

FT-TRMS

Development Pipeline & Module Prompt Bible

Field Travel Tracking & Reimbursement Management System

Total Modules	Total Sprints	Estimated Duration
9 Modules	6 Sprints	12 Weeks

Version 1.0 | 9 April 2026 | For Development Team Use

PART 1 — DEVELOPMENT PIPELINE

This pipeline maps the entire project lifecycle from environment setup to production deployment. Each sprint is self-contained with clear entry/exit criteria, deliverables, and test gates before the next sprint begins.

Phase 0: Project Bootstrap & Environment Setup

Duration: 3 days | Owner: Lead Dev | Blocker for all sprints

0.1 Repository Structure

```
ft-trms/
├── backend/           # Node.js + Express + TypeScript
├── mobile/            # React Native (Expo)
├── web/               # Next.js + Tailwind CSS
├── shared/            # Shared TypeScript types & Zod schemas
├── .github/
│   └── workflows/    # CI/CD YAML files
├── docker-compose.yml
└── README.md
```

0.2 Bootstrap Checklist

Task	Command / Action	Owner
Init monorepo	npm init -w backend -w mobile -w web -w shared	Lead
Supabase project	supabase.com → new project → copy URL + keys	Lead
PostgreSQL schema	Run migration SQL in Supabase SQL editor	Backend Dev
Redis (local dev)	docker-compose up redis	Backend Dev
Expo init	npx create-expo-app mobile --template	Mobile Dev
Next.js init	npx create-next-app web --typescript --tailwind --app	Web Dev
Shared types pkg	tsc --init in /shared; add to workspace deps	Lead
ENV files	Copy .env.example → .env for all three packages	All
Mapbox token	Add MAPBOX_ACCESS_TOKEN to mobile + web .env	Lead

Sprint 1: Foundation — Auth, DB, Project Scaffold

Duration: 2 Weeks | Team: Full Team | Parallel tracks possible after Day 3

Track	Days 1–3	Days 4–7	Days 8–10
Backend	Express app, middleware, Zod, logging	Supabase Auth integration, JWT middleware, RBAC	Users CRUD, role assignment endpoints
Mobile	Expo setup, navigation (Expo Router), Supabase client	Login screen, token storage (SecureStore), auth guard	Employee home scaffold, tab navigation
Web	Next.js setup, Tailwind, shadcn/ui install	Login page, Supabase browser client, auth guard HOC	Dashboard layout, sidebar, role-based routing
Shared	TypeScript interfaces: User, Role, Trip, Claim, Location	Zod schemas matching DB tables	API response wrapper types

Sprint 1 Exit Criteria

- Employee can register + login on mobile app
- Manager can login on web dashboard
- Admin can create users and assign roles via API (Postman tested)
- JWT middleware rejects invalid/expired tokens with 401
- All 4 roles have correct route-level access enforcement

Sprint 2: Trip Engine — GPS Tracking Core

|Duration: 2 Weeks | Most complex sprint — GPS + Socket.IO + DB writes

Track	Days 1–4	Days 5–8	Days 9–10
Backend	Trip CRUD endpoints, status machine, Haversine util	Socket.IO server, room management, location ingestion	Fraud detection middleware, speed/accuracy validation
Mobile	expo-location setup, permission flow, 15-sec watcher	Socket.IO client, location emitter, trip state machine	Trip UI: punch-in, live timer, pause/resume/end buttons
Web	Live map component (Mapbox GL), employee markers	Socket.IO client, subscribe to manager room events	Employee list sidebar with live status badges
Shared	Trip status enum, Location payload type, Socket event types	Fraud flag types, anomaly codes	—

Sprint 2 Exit Criteria

- Employee punches in → GPS tracked every 15 sec → points stored in DB
- Manager dashboard shows employee moving on map in real time (< 2s lag)

- Trip pause/resume correctly excludes paused time from total duration
- Speed > 200 km/h flags point as is_anomaly without crashing the app
- Offline: location buffered in AsyncStorage; synced on reconnect

Sprint 3: Claims & Approval Workflow

|Duration: 2 Weeks | Business logic intensive — state machine + notifications

Track	Days 1–4	Days 5–8	Days 9–10
Backend	Auto-claim creation on trip end, rate resolution logic	Approval workflow endpoints, status transition guards	Socket events for claim notifications, audit log writes
Mobile	Claim detail screen, review + submit UI, notes input	Claim history list with status chips and filter	Push notification setup (Expo Notifications)
Web	Manager claims queue table with approve/reject modal	Accountant read-only view with status filters	CSV export endpoint + download button
Shared	Claim status enum, Approval payload types	Notification event types	—

Sprint 3 Exit Criteria

- Trip end auto-creates DRAFT claim with correct amount (distance x rate)
- Employee submits claim; manager sees it in dashboard within 5 sec (socket)
- Manager approves/rejects with comments; employee notified via push
- Accountant can view only APPROVED/REJECTED claims; cannot modify any
- CSV export of claims generates correct file with all required columns

Sprint 4: Admin Panel & User Management

|Duration: 1.5 Weeks | CRUD-heavy; prioritize validation and soft-delete

Track	Days 1–4	Days 5–7
Backend	Users CRUD with Supabase Auth sync (invite email), employee-manager assignment table	System settings API (global rate, max speed), activity logs endpoint
Web (Admin)	User management table: create/edit/deactivate, role dropdown	Assignment UI: drag-drop or dropdown employee→manager mapping
Web (Analytics)	Trip count / distance charts (recharts), claims summary cards	Per-employee report: trips, distance, claim totals with date filter

Sprint 4 Exit Criteria

- Admin can create user → Supabase Auth invite email sent → user can login
- Soft-delete deactivates user without removing historical trip/claim data
- Employee-manager assignment reflected in manager's live tracking dashboard
- Analytics charts render with real data from DB (not mocked)

Sprint 5: Hardening — Security, Performance, Fraud

|Duration: 1.5 Weeks | Non-negotiable for production readiness

Area	Task	Implementation
Rate Limiting	Apply to all auth + public routes	express-rate-limit: 100/min public, 1000/min authed
Input Validation	Zod on every request body + query param	Validate middleware wrapper on all routers
SQL Safety	Audit all DB queries	No raw string interpolation; all parameterized via pg
CORS	Lock down to known origins	Whitelist Expo dev URL + production web domain only
Performance	Batch location inserts	Buffer 5 points server-side; single multi-row INSERT
DB Indexes	Verify all indexes are applied	EXPLAIN ANALYZE on top 5 slowest queries
Redis Caching	Cache live employee positions	10-sec TTL; invalidate on status change
Fraud	Mock location detection integration	Android mock provider check; iOS accuracy spike detection
Geofencing	Office punch-in zone validation	Turf.js booleanPointInPolygon on trip start

Sprint 6: Testing, CI/CD & Deployment

|Duration: 2 Weeks | Do not skip — this determines production stability

6.1 Test Coverage Targets

Layer	Test Type	Tool	Target Coverage
Backend Services	Unit tests	Jest + ts-jest	> 80% line coverage
Backend API	Integration tests	Supertest + local Supabase	All endpoints tested
Auth Middleware	Unit tests	Jest mocks	100% — security critical

Layer	Test Type	Tool	Target Coverage
Fraud Detection	Unit tests	Jest	All anomaly cases covered
Mobile Flows	E2E (optional MVP)	Detox	Happy path: login → trip → claim
Web Dashboard	Component tests	React Testing Library	Key components: MapView, ClaimsTable

6.2 CI/CD Pipeline (GitHub Actions)

```
# .github/workflows/backend.yml
on: [push, pull_request]
jobs:
  test:
    steps:
      - uses: actions/checkout@v4
      - run: npm ci
      - run: npm run lint
      - run: npm run typecheck
      - run: npm run test:coverage # fail if < 80%
  build:
    needs: test
    if: github.ref == 'refs/heads/main'
    steps:
      - run: docker build -t ft-trms-api .
      - run: docker push $ECR_REGISTRY/ft-trms-api:$SHA
  deploy-staging:
    needs: build
    steps:
      - run: aws ecs update-service --force-new-deployment
  deploy-prod:
    needs: deploy-staging
    environment: production # requires manual approval
    steps:
      - run: aws ecs update-service --cluster prod ...
```

6.3 Deployment Targets

Service	Platform	Plan / Config
Backend API	Render (Web Service) or AWS ECS Fargate	512 MB RAM, 0.25 vCPU; auto-scale to 4 instances
Web Dashboard	Vercel	Free tier; automatic branch previews
Database	Supabase	Pro plan: 8 GB DB, daily backups, read replica
Redis	Upstash (serverless) or AWS ElastiCache	Upstash for MVP: free 10K req/day; ElastiCache at scale
File Storage	Supabase Storage	Receipt photos, max 5MB per file
Mobile App	Expo EAS Build	eas build --platform all; submit to App Store + Play Store

Cross-Cutting Development Standards

Branching Strategy (Git Flow)

```
main      ← production only; protected; requires PR + review
develop   ← integration branch; all features merge here first
feature/* ← one branch per task (e.g. feature/trip-pause-resume)
fix/*     ← bug fixes
release/* ← pre-release stabilization

PR Rules:
- Minimum 1 reviewer approval required
- CI must pass (lint + typecheck + tests)
- No direct pushes to main or develop
```

Commit Message Convention (Conventional Commits)

Type	Usage	Example
feat	New feature	feat(trips): add pause/resume endpoint
fix	Bug fix	fix(auth): handle expired refresh token edge case
chore	Config / tooling	chore: add eslint rule for no-console
test	Tests only	test(claims): add approval workflow unit tests
docs	Documentation	docs: update API endpoint table in README
perf	Performance	perf(locations): batch insert 5 points per DB call
security	Security fix	security(auth): add rate limit to login endpoint

Definition of Done (Every Task)

- Code written and self-reviewed (no console.log, no TODOs)
- TypeScript: zero type errors (tsc --noEmit passes)
- ESLint: zero warnings
- Unit tests written for business logic functions
- API changes reflected in Postman collection (shared team workspace)
- PR description explains what changed and why
- Reviewed and approved by at least one teammate

PART 2 — MODULE DEVELOPMENT PROMPTS

Each module below contains a self-contained prompt designed to be given directly to an AI coding assistant (Claude, Cursor, GitHub Copilot Chat, etc.) or used as a detailed specification for a developer. The prompts include full context, constraints, expected outputs, and acceptance criteria so each module can be developed independently.

How to use: Copy the prompt block for a module and paste it into your AI assistant. Include the shared/types.ts file content for best results.

Module Map

#	Module Name	Platform	Sprint	Dependencies
M1	Project Scaffold & Shared Types	All	Sprint 1	None — build first
M2	Auth Service	Backend + Mobile + Web	Sprint 1	M1, Supabase project
M3	Trip Engine (CRUD + State Machine)	Backend + Mobile	Sprint 2	M1, M2
M4	GPS Tracking & Realtime (Socket.IO)	Backend + Mobile + Web	Sprint 2	M3
M5	Claims & Approval Workflow	Backend + Mobile + Web	Sprint 3	M3
M6	Admin Panel & User Management	Backend + Web	Sprint 4	M2
M7	Maps & Route Visualisation	Mobile + Web	Sprint 2–3	M4, Mapbox key
M8	Fraud Detection & Validation	Backend	Sprint 5	M4
M9	Analytics Dashboard	Web	Sprint 4–5	M3, M5, M6

M1 — Project Scaffold & Shared Types

Platform	All (Backend, Mobile, Web)
Sprint	Sprint 1, Days 1–3
Output	Monorepo scaffold + /shared/types package with all TypeScript interfaces and Zod schemas
Blocks	ALL other modules — must be completed first

Prompt for AI Assistant / Developer

|PASTE THIS ENTIRE BLOCK into your AI assistant as the first message:

```
You are helping me build the FT-TRMS (Field Travel Tracking &
Reimbursement Management System) – a GPS-based field employee
tracking and expense reimbursement platform.

TASK: Set up the monorepo scaffold and create the /shared package
containing all TypeScript types and Zod validation schemas.

MONOREPO STRUCTURE (npm workspaces):
ft-trms/
├── backend/      (Node.js + Express + TypeScript)
├── mobile/       (React Native / Expo)
├── web/          (Next.js 14 App Router + Tailwind CSS)
├── shared/       (shared types + Zod schemas – THIS MODULE)
└── package.json (root workspace config)

SHARED PACKAGE MUST EXPORT:

1. ENUMS:
  - UserRole: EMPLOYEE | MANAGER | ADMIN | ACCOUNTANT
  - TripStatus: STARTED | PAUSED | RESUMED | ENDED | CANCELLED
  - ClaimStatus: DRAFT | SUBMITTED | APPROVED | REJECTED | PAID
  - ApprovalAction: APPROVED | REJECTED
  - AnomalyReason: SPEED_ANOMALY | TELEPORT | LOW_ACCURACY | MOCK_LOCATION

2. INTERFACES (match the PostgreSQL schema exactly):
  - User, Trip, LocationPoint, Claim, ClaimApproval
  - EmployeeManagerAssignment, SystemSetting

3. API TYPES:
  - ApiResponse<T> = { success: boolean; data?: T; error?: string; meta?: PaginationMeta }
  - PaginationMeta = { page, limit, total, totalPages }
  - AuthTokens = { access_token: string; refresh_token: string; expires_in: number }

4. SOCKET EVENT PAYLOADS:
  - LocationUpdatePayload: { trip_id, latitude, longitude, accuracy_m?, speed_kmh?, ts:
string }
  - EmployeeMovedPayload: { employee_id, employee_name, lat, lng, ts, speed_kmh?, trip_id
}
  - TripStatusPayload: { trip_id, employee_id, status: TripStatus }
  - ClaimNotificationPayload: { claim_id, employee_id, amount, status: ClaimStatus }

5. ZOD SCHEMAS (for API validation):
  - LoginSchema: { email: string (email), password: string (min 8) }
```

```
- StartTripSchema: { latitude, longitude, device_id?, notes? }  
- LocationPointSchema: { latitude (-90 to 90), longitude (-180 to 180),  
    accuracy_m?, speed_kmh?, recorded_at: ISO string }  
- SubmitClaimSchema: { notes? }  
- ApproveClaimSchema: { comments? }  
- RejectClaimSchema: { comments: string (min 10, required) }  
- CreateUserSchema: { email, full_name, role: UserRole, phone?, rate_per_km? }
```

CONSTRAINTS:

- Use TypeScript strict mode
- All interfaces must have JSDoc comments
- Export everything from a single index.ts
- Zod schemas must export both the schema AND the inferred type
- Do NOT include any runtime logic – types and schemas only

OUTPUT: Complete /shared/src/index.ts with all exports,
/shared/package.json, and /shared/tsconfig.json

M2 — Auth Service

Platform	Backend (Express) + Mobile (Expo) + Web (Next.js)
Sprint	Sprint 1, Days 4–10
Auth Provider	Supabase Auth (JWT RS256, access_token 15min, refresh 30 days)
Output	Working login/logout on all 3 platforms with RBAC enforcement

M2-A: Backend Auth Middleware — Prompt

```

You are building the authentication and authorization layer for FT-TRMS.
Tech: Node.js + Express + TypeScript + Supabase Auth + @supabase/supabase-js v2

BUILD THE FOLLOWING FILES:

1. src/config/supabase.ts
  - Init Supabase client with SERVICE_ROLE_KEY (server-side only)
  - Export: supabaseAdmin (uses service role key)
  - Never expose service key to client

2. src/middleware/authenticate.ts
  - Express middleware that reads Authorization: Bearer <token>
  - Calls supabaseAdmin.auth.getUser(token) to verify
  - On success: fetches user record from users table (not just auth)
  - Attaches to req.user = { id, email, role, full_name, rate_per_km }
  - On failure: returns 401 { success: false, error: 'Unauthorized' }
  - Handle edge cases: missing header, malformed token, revoked session

3. src/middleware/requireRole.ts
  - Factory: requireRole(...roles: UserRole[]) => RequestHandler
  - Checks req.user.role is in allowed roles
  - Returns 403 if not: { success: false, error: 'Insufficient permissions' }
  - MUST be used AFTER authenticate middleware

4. src/modules/auth/auth.router.ts
  - POST /auth/login
    Body: { email, password }
    Action: supabase.auth.signInWithPassword()
    Response: { access_token, refresh_token, user: { id, email, role, full_name } }
  - POST /auth/refresh
    Body: { refresh_token }
    Action: supabase.auth.refreshSession()
    Response: new { access_token, refresh_token }
  - POST /auth/logout
    Auth: required
    Action: supabase.auth.signOut()
  - GET /auth/me
    Auth: required
    Response: req.user object

5. src/middleware/rateLimiter.ts
  - Use express-rate-limit
  - authLimiter: 5 attempts per 15 min per IP (for login route)
  - apiLimiter: 200 requests per minute per IP (for all other routes)

TYPES: Import UserRole, ApiResponse, AuthTokens from '@ft-trms/shared'
ERROR HANDLING: All errors wrapped in try/catch; use a custom AppError class

```

```

LOGGING: Use winston logger; log auth events (login success/fail, logout)
TESTS: Write Jest unit tests for authenticate middleware covering:
- valid token → attaches req.user
- missing token → 401
- expired token → 401 with specific message
- valid token but user not in users table → 401

```

M2-B: Mobile Auth — Prompt

You are building the authentication flow for the FT-TRMS mobile app.
 Tech: React Native (Expo) + TypeScript + @supabase/supabase-js v2
 + expo-secure-store + Expo Router (file-based routing)

BUILD THE FOLLOWING:

1. lib/supabase.ts
 - Init Supabase with ANON_KEY (from app.config.js / Constants.expoConfig)
 - Configure AsyncStorage as session storage for React Native
 - Export: supabase client
2. store/authStore.ts (Zustand)
 - State: { user: User | null, session: Session | null, loading: boolean }
 - Actions: login(email, password), logout(), refreshSession(), initialize()
 - initialize(): called on app start; restores session from SecureStore
 - On login success: store access_token in expo-secure-store
 - On logout: clear SecureStore + Supabase session
3. app/(auth)/login.tsx
 - Form: email input, password input, login button
 - Validation: use LoginSchema from @ft-trms/shared (Zod)
 - Loading state on submit
 - Error display: inline under relevant field
 - On success: router.replace('/(employee)/home')
4. app/_layout.tsx (Root Layout)
 - On mount: call authStore.initialize()
 - If no session → redirect to /(auth)/login
 - If session exists → redirect to role-based home:
 - EMPLOYEE → /(employee)/home
 - (managers/admins use web dashboard, mobile is employee-only)
5. services/api.ts
 - Axios instance with baseURL from env
 - Request interceptor: attach Authorization: Bearer <access_token>
 - Response interceptor: on 401 → call refreshSession() → retry once
 - On refresh fail: clear session → redirect to login

IMPORTANT:

- Never store access_token in AsyncStorage (use SecureStore)
- Handle network errors gracefully with user-visible toast messages
- Loading skeleton on app initialize (not blank screen)

M2-C: Web Dashboard Auth — Prompt

You are building auth for the FT-TRMS web dashboard.
 Tech: Next.js 14 (App Router) + TypeScript + @supabase/ssr + Tailwind CSS

BUILD THE FOLLOWING:

1. lib/supabase/client.ts - browser Supabase client (createBrowserClient)

```
2. lib/supabase/server.ts - server Supabase client (createServerClient + cookies)
3. middleware.ts (Next.js middleware)
  - Run on all routes except /login and /api/auth/*
  - Verify session via server client
  - Redirect unauthenticated → /login
  - Redirect authenticated user to correct dashboard by role:
    MANAGER → /dashboard/manager
    ADMIN → /dashboard/admin
    ACCOUNTANT → /dashboard/accountant

4. app/(auth)/login/page.tsx
  - Clean login form with email + password
  - Uses Supabase browser client signInWithPassword
  - After login: router.push(roleBasedPath)
  - Show error toast on failure

5. hooks/useAuth.ts
  - Returns: { user, role, loading, logout }
  - Subscribes to onAuthStateChange

6. components/RoleGuard.tsx
  - Client component wrapping protected page sections
  - Props: allowedRoles: UserRole[]
  - If user.role not in allowedRoles → render AccessDenied component

ROLE-BASED LAYOUT:
- /dashboard/manager/* → Manager layout (live map sidebar visible)
- /dashboard/admin/* → Admin layout (user management nav)
- /dashboard/accountant/* → Accountant layout (claims only, read-only badge)
```

M3 — Trip Engine (CRUD + State Machine)

Platform	Backend + Mobile
Sprint	Sprint 2, Days 1–4
Core Logic	Trip state machine + Haversine distance + route storage

M3-A: Backend Trip Service — Prompt

```
You are building the Trip Service for FT-TRMS.
Tech: Node.js + Express + TypeScript + node-postgres (pg) + Supabase

TRIP STATE MACHINE:
  STARTED → PAUSED → RESUMED → ENDED
  STARTED → ENDED (no pause)
  Any state → CANCELLED (admin only)
  Invalid transitions must return 409 Conflict

BUILD THE FOLLOWING:

1. src/modules/trips/trip.service.ts
  Functions:
  - startTrip(employeeId, payload: StartTripSchema)
    Creates trip record with status STARTED
    Records start lat/lng as first LocationPoint
    Returns trip with socket_room name ('trip:{id}')

  - pauseTrip(tripId, employeeId)
    Validates: trip exists, belongs to employee, status is STARTED|RESUMED
    Updates: status=PAUSED, stores pause_start_at in Redis with key 'pause:{tripId}'

  - resumeTrip(tripId, employeeId)
    Validates: status is PAUSED
    Gets pause_start_at from Redis
    Updates: status=RESUMED, adds elapsed seconds to paused_duration_sec

  - endTrip(tripId, employeeId)
    Validates: status is STARTED|RESUMED|PAUSED
    Calls calculateTripDistance(tripId) → sums Haversine segments
    Updates: status=ENDED, total_distance_km, end_time
    Calls ClaimService.autoCreateClaim(tripId) — MUST be atomic (transaction)

  - calculateTripDistance(tripId): number
    SELECT ordered location_points for trip
    Apply Haversine between consecutive points
    Skip points where is_anomaly=true
    Return total km

2. src/utils/haversine.ts
  - haversine(a: {lat,lng}, b: {lat,lng}): number — returns km
  - Must handle edge cases: same point → 0, antipodal points
  - Export unit tests: write them in __tests__/haversine.test.ts

3. src/modules/trips/trip.router.ts
  POST /trips/start → authenticate, requireRole(EMPLOYEE)
  POST /trips/:id/pause → authenticate, requireRole(EMPLOYEE)
  POST /trips/:id/resume → authenticate, requireRole(EMPLOYEE)
  POST /trips/:id/end → authenticate, requireRole(EMPLOYEE)
```

```

GET    /trips                → authenticate, requireRole(EMPLOYEE) – own trips
GET    /trips/active         → authenticate, requireRole(MANAGER,ADMIN)
GET    /trips/:id           → authenticate (owner OR manager of owner OR admin)
GET    /trips/:id/route      → authenticate, requireRole(MANAGER,ADMIN)

```

4. OWNERSHIP CHECK MIDDLEWARE:

- tripBelongsToEmployee(req, res, next)
- Verifies trip.employee_id === req.user.id
- Manager can access trips of their assigned employees only

DATABASE: All DB calls use parameterized queries (\$1, \$2) via pg Pool

TRANSACTIONS: endTrip uses BEGIN/COMMIT to ensure trip update + claim creation is atomic

TESTS: Jest unit tests for state machine transitions (valid + invalid cases)

M3-B: Mobile Trip UI — Prompt

You are building the trip lifecycle UI for the FT-TRMS employee mobile app.

Tech: React Native (Expo) + TypeScript + Expo Router + Zustand + Axios

BUILD THE FOLLOWING:

- store/tripStore.ts (Zustand)
 - State:
 - activeTripId: string | null
 - tripStatus: TripStatus | null
 - startTime: Date | null
 - pausedAt: Date | null
 - totalPausedMs: number
 - elapsedSeconds: number (live counter, updated every second)
 - Actions:
 - startTrip(payload) → API call → set state
 - pauseTrip() → API call → record pausedAt
 - resumeTrip() → API call → add to totalPausedMs
 - endTrip() → API call → clear state → navigate to claim review
 - tick() → called every second by interval → updates elapsedSeconds
- app/(employee)/home.tsx
 - If no active trip: show large 'Punch In' button (green)
 - PunchIn modal: notes text input + confirm button
 - If STARTED: show live timer (HH:MM:SS), 'Pause' and 'End Trip' buttons
 - If PAUSED: show 'Paused' indicator, 'Resume' and 'End Trip' buttons
 - Timer excludes paused duration in real time
- components/trip/TripTimer.tsx
 - Accepts elapsedSeconds: number
 - Displays HH:MM:SS format
 - Animates: subtle pulse when STARTED, grey when PAUSED
- components/trip/TripActionBar.tsx
 - Shows correct buttons based on tripStatus
 - Loading state on each button while API is in flight
 - Disable all buttons if network is offline (show offline banner)
- app/(employee)/trip-history.tsx
 - Paginated FlatList of past trips
 - Each card: date, distance, duration, claim status badge
 - Pull-to-refresh
 - Tap → navigate to trip detail screen

UX RULES:

- End Trip requires confirmation dialog (cannot be undone)
- If app is killed mid-trip, tripStore rehydrates from AsyncStorage on launch

- All API errors shown as toast notifications (react-native-toast-message)

M4 — GPS Tracking & Realtime (Socket.IO)

Platform	Backend + Mobile (emitter) + Web (receiver)
Sprint	Sprint 2, Days 5–10
GPS Interval	15 seconds (dynamic: 5s when speed > 30 km/h, 30s when stationary)

M4-A: Backend Socket.IO Server — Prompt

```

You are building the realtime tracking server for FT-TRMS.
Tech: Node.js + Socket.IO v4 + @socket.io/redis-adapter + ioredis + TypeScript

BUILD src/realtime/ directory:

1. src/realtime/socket.server.ts
  - Create Socket.IO server attached to Express HTTP server
  - Configure Redis adapter: io.adapter(createAdapter(pubClient, subClient))
  - Auth middleware on connection:
    socket.handshake.auth.token → verify JWT → attach socket.data.user
    Reject with 'Unauthorized' if invalid
  - CORS: same origins as REST API

2. src/realtime/handlers/employee.handler.ts
  Events to handle (employee sockets):

  ON 'join_trip' ({ trip_id })
    - Verify trip belongs to this employee
    - socket.join('trip:' + trip_id)
    - Find manager of this employee from DB
    - Emit 'employee_online' to 'manager:{managerId}' room

  ON 'location_update' (LocationUpdatePayload)
    - Call FraudDetectionService.validate(prev, curr) — get from Redis
    - If valid: INSERT into location_points
      Store latest position in Redis: SET 'live:{employeeId}' JSON 60s TTL
      Emit 'employee_moved' to rooms: 'manager:{managerId}' AND 'admin:global'
    - If anomaly: INSERT with is_anomaly=true, do NOT broadcast
      Emit 'anomaly_detected' to 'admin:global'
    - Acknowledge to client: callback({ received: true, anomaly: false })

  ON 'trip_status' ({ trip_id, status })
    - Emit 'trip_status_changed' to manager and admin rooms

  ON 'disconnect'
    - Set employee status offline in Redis
    - Notify manager room

3. src/realtime/handlers/manager.handler.ts
  ON 'subscribe_team'
    - Fetch assigned employee IDs for this manager
    - socket.join('manager:' + managerId)
    - Return current live positions from Redis for all assigned employees

4. src/realtime/services/liveLocation.service.ts
  - getLivePosition(employeeId): { lat, lng, ts } | null — reads from Redis
  - getAllTeamPositions(managerId): array of live positions for team

5. EMIT CONTRACT (server → client events):

```

```
'employee_moved': EmployeeMovedPayload
'trip_status_changed': TripStatusPayload
'claim_submitted': ClaimNotificationPayload
'claim_actioned': { claim_id, status, comments }
'employee_online': { employee_id }
'employee_offline': { employee_id }
'anomaly_detected': { employee_id, trip_id, reason: AnomalyReason }

IMPORT ALL PAYLOAD TYPES from '@ft-trms/shared'
LOG all socket events using winston at 'debug' level in dev, 'info' in prod
```

M4-B: Mobile GPS Emitter — Prompt

You are building the GPS tracking engine for the FT-TRMS mobile app.
 Tech: React Native (Expo) + expo-location + expo-task-manager + socket.io-client

BUILD THE FOLLOWING:

- tasks/locationTask.ts (Background TaskManager task)
 - Task name: 'BACKGROUND_LOCATION_TASK'
 - On new location: if socket connected → emit 'location_update'
 - If offline: push to locationQueue (AsyncStorage buffer, max 500 entries)
 - Adaptive interval logic:
 - speed > 30 km/h → setInterval 5000ms
 - speed 5-30 km/h → setInterval 15000ms (default)
 - speed < 5 km/h → setInterval 30000ms
- hooks/useLocation.ts
 - requestPermissions(): checks + requests FOREGROUND and BACKGROUND
 - startForegroundTracking(tripId): watchPositionAsync with Accuracy.Balanced
 - startBackgroundTracking(tripId): startLocationUpdatesAsync
 - stopTracking(): removes both watchers
 - Attach current tripId to all emitted payloads
- hooks/useSocket.ts
 - Connect to backend WS on app start (after login)
 - Auth: { token: accessToken } in handshake
 - Auto-reconnect: exponential backoff (max 5 retries, 2s initial delay)
 - Expose: socket, connected, emit, on, off
 - On reconnect: flush locationQueue buffer via POST /trips/:id/bulk-locations
- services/locationQueue.ts
 - enqueue(point: LocationPointSchema): append to AsyncStorage list
 - flush(tripId): POST all queued points to /trips/:id/bulk-locations → clear queue
 - getCount(): returns number of queued points

BACKGROUND TRACKING SETUP (app.config.js):
 iOS: UIBackgroundModes: ['location'], NSLocationAlwaysUsageDescription
 Android: ACCESS_BACKGROUND_LOCATION, FOREGROUND_SERVICE permissions

BATTERY OPTIMIZATION:

- distanceInterval: 20 (metres) to skip stationary duplicate points
- accuracy: Accuracy.Balanced (NOT Accuracy.High) in background
- Stop background tracking immediately on trip END

M4-C: Web Live Map — Prompt

You are building the live employee tracking map for the FT-TRMS manager dashboard.
 Tech: Next.js 14 + TypeScript + mapbox-gl + socket.io-client + Zustand
 Mapbox token: available as process.env.NEXT_PUBLIC_MAPBOX_TOKEN

BUILD THE FOLLOWING:

1. store/liveTrackingStore.ts (Zustand)

```
State:
- employees: Map<employeeId, { name, lat, lng, ts, status, trip_id }>
Actions:
- initFromServer(positions[]): populate from initial REST call
- updatePosition(EmployeeMovedPayload): upsert employee position
- updateStatus(TripStatusPayload): update employee status
- setOffline(employeeId): mark as offline
```

2. components/maps/LiveTrackingMap.tsx

```
- Initialize Mapbox GL map (style: 'mapbox://styles/mapbox/streets-v12')
- Display employee markers using custom HTML markers (not GL symbols)
- Marker: circle with employee initials; green = ACTIVE, amber = PAUSED,
  grey = OFFLINE
- On employee position update from store: smoothly animate marker to new position
  using map.easeTo() - NOT an instant jump
- Clicking a marker: shows popup with { name, speed, last_seen, trip_id link }
- Fit bounds to show all active employees on first load
```

3. hooks/useManagerSocket.ts

```
- Connect socket with manager JWT
- On connect: emit 'subscribe_team'
- Listen for 'employee_moved' → store.updatePosition()
- Listen for 'trip_status_changed' → store.updateStatus()
- Listen for 'employee_online/offline' → update store
- Listen for 'claim_submitted' → show toast notification
```

4. components/maps/RouteReplayMap.tsx

```
- Accepts: tripId prop
- Fetches GET /trips/:id/route on mount
- Draws GeoJSON LineString polyline in blue on the map
- Adds start marker (green pin) and end marker (red pin)
- Fits map bounds to the route
- Shows anomaly points as orange warning markers
```

IMPORTANT:

```
- mapbox-gl must be imported dynamically (no SSR): dynamic(() => import(...), { ssr: false
})
- Map container must have explicit height (not 100% of undefined parent)
```

M5 — Claims & Approval Workflow

Platform	Backend + Mobile + Web
Sprint	Sprint 3
Core Logic	Auto-generate on trip end → Employee submits → Manager approves/rejects → Accountant views

M5 Combined Prompt (Backend + Web + Mobile)

You are building the Claims & Approval workflow for FT-TRMS.
This module spans Backend, Mobile, and Web.

BACKEND — src/modules/claims/

1. claim.service.ts

```

autoCreateClaim(tripId: string): Promise<Claim>
- Called inside Trip.endTrip() transaction
- Fetch trip.total_distance_km
- Resolve rate: employee.rate_per_km ?? system_settings['global_rate_per_km']
- amount = ROUND(distance * rate, 2)
- INSERT into claims: status=DRAFT, snapshot the rate (immutable audit trail)
- Return created claim

submitClaim(claimId, employeeId, notes?)
- Validate: claim.status === DRAFT, claim.employee_id === employeeId
- UPDATE status → SUBMITTED
- Emit Socket.IO event 'claim_submitted' to manager room

approveClaim(claimId, managerId, comments?)
- Validate: claim.status === SUBMITTED
- Verify: manager is assigned to this employee
- UPDATE claims.status → APPROVED
- INSERT claim_approvals { claim_id, reviewed_by, action: APPROVED, comments }
- Emit 'claim_actioned' to employee socket

rejectClaim(claimId, managerId, comments: string)
- Validate: claim.status === SUBMITTED, comments.length >= 10 (required)
- UPDATE claims.status → REJECTED
- INSERT claim_approvals record
- Emit 'claim_actioned' to employee socket

exportClaimsCSV(filters): string (CSV content)
- Joins claims + users + trips
- Columns: claim_id, employee_name, trip_date, distance_km, rate, amount,
           status, submitted_at, acted_at, manager_comments
- Use fast-csv or json2csv library

```

2. claim.router.ts

```

GET  /claims           → EMPLOYEE (own claims, paginated, filter by status)
GET  /claims/team      → MANAGER (assigned employees, filter by status)
GET  /claims/all       → ACCOUNTANT | ADMIN (all, filter, CSV export)
GET  /claims/:id       → owner | manager | accountant | admin
POST /claims/:id/submit → EMPLOYEE
POST /claims/:id/approve → MANAGER

```

```
POST /claims/:id/reject    → MANAGER
GET  /claims/export/csv    → ACCOUNTANT | ADMIN – returns CSV file download
```

MOBILE – Claims Screens

app/(employee)/claims/index.tsx

- Tab screen: 'My Claims'
- FlatList of ClaimCard components, grouped by status
- Filter chips: ALL | DRAFT | SUBMITTED | APPROVED | REJECTED
- Each card: trip date, km, amount (bold), status badge (color-coded)

app/(employee)/claims/[id].tsx

- Claim detail: trip summary (date, distance, duration)
- Calculation breakdown: {distance} km x Rs.{rate} = Rs.{amount}
- If DRAFT: notes text input + 'Submit Claim' button
- If REJECTED: show manager comments in red callout box
- If APPROVED: show green success state

WEB – Manager Claims Queue

app/dashboard/manager/claims/page.tsx

- DataTable with columns: Employee, Trip Date, Distance, Amount, Submitted Date, Status, Actions
- Filter bar: status dropdown, employee search, date range picker
- Row actions: 'Approve' (green) | 'Reject' (red) buttons
- Approve: inline confirmation with optional comments textarea
- Reject: modal with REQUIRED comments (min 10 chars), character counter
- Optimistic UI: update row status immediately; revert on API error

app/dashboard/accountant/claims/page.tsx

- Same table but NO action buttons – read-only
- Shows only APPROVED and REJECTED claims
- 'Export CSV' button → GET /claims/export/csv → triggers browser download
- Summary cards: Total Approved Amount (month), Count by status

SHARED CONSTRAINT: All monetary amounts displayed with Indian formatting:
Intl.NumberFormat('en-IN', { style: 'currency', currency: 'INR' })

M6 — Admin Panel & User Management

Platform	Backend + Web (Admin role only)
Sprint	Sprint 4

M6 Prompt

You are building the Admin Panel for FT-TRMS. Only ADMIN role can access.
Tech: Backend (Express + TypeScript), Web (Next.js + shadcn/ui + Tailwind)

=====

BACKEND

=====

```
1. user.service.ts

createUser(payload: CreateUserSchema)
- Call supabaseAdmin.auth.admin.inviteUserByEmail(email)
  This sends invite email; user sets own password
- On auth user created (via webhook or after invite): INSERT into users table
- Set role, full_name, phone, rate_per_km

updateUser(userId, patch)
- Can update: full_name, phone, role, rate_per_km, is_active
- If is_active set to false: also call supabaseAdmin.auth.admin.updateUserById
  to ban the user (they cannot login while inactive)

assignManager(employeeId, managerId)
- Validate: employee role=EMPLOYEE, manager role=MANAGER
- Deactivate existing active assignment for this employee
- INSERT new employee_manager_assignments record

getSystemSettings(): Record<string, string>
updateSystemSetting(key, value, adminId): updates system_settings table
Key settings: 'global_rate_per_km', 'max_speed_kmh', 'gps_interval_sec'

2. user.router.ts (all routes: authenticate + requireRole(ADMIN))
GET    /users                paginated, filter by role/is_active
POST   /users                create + invite
PATCH /users/:id            update
DELETE /users/:id            soft delete (is_active=false)
POST   /users/:id/assign-manager body: { manager_id }
GET    /users/:id/summary    trips count, total distance, claims total
GET    /settings             get all system settings
PATCH /settings/:key        update a setting
```

=====

WEB — Admin Dashboard

=====

```
app/dashboard/admin/users/page.tsx
- DataTable: Name, Email, Role badge, Status (Active/Inactive), Manager, Actions
- 'New User' button → slide-over form panel (not modal)
- Inline role badge edit (dropdown on click)
- Deactivate: confirmation dialog with warning about impact
- Search: debounced email/name search

app/dashboard/admin/assignments/page.tsx
```

- Two-column layout: left = unassigned employees, right = manager groups
- Drag employee card from left into a manager group on right
- Alternatively: dropdown select for each employee row
- Shows current assignment with assigned date

app/dashboard/admin/settings/page.tsx

- Form: Global Rate per KM (number input with Rs. prefix)
- Max Speed Threshold for fraud (km/h)
- GPS Interval (seconds, 5-60 range)
- Save button with optimistic update + toast

app/dashboard/admin/analytics/page.tsx

- Summary cards: Total Employees, Active Trips Today, Claims This Month, Total Reimbursed (INR)
- Bar chart: Daily trip count (last 30 days)
- Line chart: Monthly reimbursement amount (last 6 months)
- Top 5 employees by distance (this month) – leaderboard table

M7 — Maps & Route Visualisation

Platform	Mobile (Mapbox RN) + Web (Mapbox GL JS)
Sprint	Sprint 2–3 (parallel with M4)
Token	MAPBOX_ACCESS_TOKEN — set in .env, never hardcode

M7 Prompt

You are building all map-related components for FT-TRMS.
Mapbox token is available as env variable. NEVER hardcode it.

MOBILE — @rnmapbox/maps

- components/maps/TripMapView.tsx
 Props: { tripId: string, isLive: boolean }

 LIVE mode (isLive=true, during active trip):
 - Show current GPS position as animated pulsing blue dot
 - Draw route polyline as points accumulate (from locationStore)
 - Camera follows current position (followUserLocation=true)
 - Show distance traveled as overlay label

 REPLAY mode (isLive=false, viewing history):
 - Fetch GET /trips/:id/route on mount
 - Draw complete route as LineLayer (blue, width 4)
 - Start marker: green MapPin at first point
 - End marker: red MapPin at last point
 - Anomaly markers: orange warning icons at is_anomaly=true points
 - Camera: fitBounds to show entire route with 50px padding
- store/locationStore.ts (Zustand)
 - points: Array<{lat, lng, ts}> — accumulated during live trip
 - addPoint(point): append to array
 - clearPoints(): on trip end
 - getGeoJSON(): convert to GeoJSON LineString for Mapbox
- MAPBOX SETUP:
 - Mapbox.setAccessToken(MAPBOX_ACCESS_TOKEN) in app entry point
 - Map style: Mapbox.StyleURL.Street
 - Handle: location permission denied → show banner, disable map

WEB — mapbox-gl (imported dynamically, no SSR)

- components/maps/LiveTrackingMap.tsx (see M4-C above)
 - Already specified in M4. Ensure smooth marker animation.
- components/maps/TripRouteMap.tsx
 Props: { tripId: string }
 - Fetches route on mount
 - Renders: LineString polyline (blue, 3px, 80% opacity)
 - Start: green circle marker
 - End: red circle marker
 - Anomalies: orange triangle markers with tooltip on hover

- Route info panel overlay: total distance, duration, anomaly count
- 'Full Screen' toggle

3. utils/mapHelpers.ts

- routeToGeoJSON(points: LocationPoint[]): GeoJSON FeatureCollection
- getBounds(points): mapboxgl.LngLatBounds – for fitBounds
- formatCoord(lat, lng): '18.5204, 73.8567' – for display

MAP INTERACTION STANDARDS (both platforms):

- Double tap/click to zoom in
- Show scale bar (miles/km toggle)
- Attribution: Mapbox and OpenStreetMap (required by Mapbox ToS)
- Loading skeleton while tiles load (grey placeholder)

M8 — Fraud Detection & Validation

Platform	Backend only (+ mobile mock detection hook)
Sprint	Sprint 5
Config	All thresholds read from system_settings table (not hardcoded)

M8 Prompt

You are building the fraud detection layer for FT-TRMS.
All thresholds come from system_settings table – never hardcode them.

BUILD src/modules/fraud/

1. fraud.service.ts

```
interface ValidationResult {
  valid: boolean;
  anomalyReason?: AnomalyReason;
  shouldStore: boolean; // true even for anomalies (for audit trail)
}

validateLocationPoint(prev: LocationPoint | null, curr: IncomingPoint):
ValidationResult

Checks (in order):

a) FIRST POINT: if prev is null → always valid (no comparison possible)

b) ACCURACY CHECK:
  if curr.accuracy_m > settings.max_accuracy_m (default 150)
  → return { valid: false, anomalyReason: LOW_ACCURACY, shouldStore: false }
  (don't store inaccurate points at all)

c) SPEED CHECK:
  distKm = haversine(prev, curr)
  timeSec = (curr.recorded_at - prev.recorded_at) / 1000
  speedKmh = (distKm / timeSec) * 3600
  if speedKmh > settings.max_speed_kmh (default 200)
  → return { valid: false, anomalyReason: SPEED_ANOMALY, shouldStore: true }

d) TELEPORT CHECK:
  if distKm > settings.max_jump_kmh (default 50) in one interval
  → return { valid: false, anomalyReason: TELEPORT, shouldStore: true }

e) ALL CHECKS PASSED:
  return { valid: true, shouldStore: true }
```

2. geofence.service.ts

```
validateTripStart(lat, lng, employeeId): Promise<GeofenceResult>
- Fetch active geofences for employee's office from DB
- Use turf.booleanPointInPolygon(point, polygon)
- If no geofences configured: return { allowed: true } (feature is optional)
- If outside all geofences: return { allowed: false, nearestOffice, distanceKm }
```

3. fraud.alerts.ts

```
- alertAdmin(tripId, employeeId, reason, details)
```

- Emits 'anomaly_detected' via Socket.IO to 'admin:global' room
- Inserts into fraud_alerts table (optional – create table in migration)
- If anomaly count for a trip > threshold (5): auto-flag trip for review

4. MOBILE SIDE – hooks/useMockLocationDetection.ts

Android only:

- Use react-native-device-info or direct NativeModule
- Check LocationManager.isFromMockProvider() on each GPS fix
- If mock detected: include { is_mock: true } in location payload
- Backend adds MOCK_LOCATION anomaly flag on this field

iOS: accuracy < 5m AND speed exactly 0 is suspicious – flag for review

TESTS (Jest – 100% coverage required for fraud logic):

- Speed anomaly: 500 km/h → flagged
- Normal speed: 60 km/h → valid
- Teleport: 100 km in 15 sec → flagged
- First point (no prev): always valid
- Low accuracy: 200m accuracy → rejected, not stored
- Geofence: point inside polygon → allowed
- Geofence: point outside polygon → denied with distance

M9 — Analytics Dashboard

Platform	Web (Admin view) + Backend (aggregation endpoints)
Sprint	Sprint 4–5
Charts	recharts library (already in Next.js)

M9 Prompt

```
You are building the Analytics module for FT-TRMS admin dashboard.
Tech: Next.js + recharts + Tailwind. Backend: Express + pg aggregation queries.

=====
BACKEND — src/modules/analytics/
=====

analytics.router.ts (all routes: ADMIN only)

GET /analytics/summary
Returns: {
  total_employees: number,
  active_trips_now: number,           -- COUNT trips WHERE status IN (STARTED,PAUSED,RESUMED)
  claims_this_month: number,
  reimbursed_this_month: number,     -- SUM claims.amount WHERE status=APPROVED + month
  avg_distance_per_trip: number,
}

GET /analytics/trips/daily?days=30
Returns: Array<{ date: string, trip_count: number, total_distance_km: number }>
SQL: GROUP BY DATE(start_time) for last N days

GET /analytics/claims/monthly?months=6
Returns: Array<{ month: string, approved_count, rejected_count, total_amount }>
SQL: GROUP BY DATE_TRUNC('month', submitted_at)

GET /analytics/employees/leaderboard?period=month&limit=10
Returns: Array<{ employee_id, name, trip_count, total_distance_km, total_claimed }>
SQL: JOIN users + trips + claims, GROUP BY employee

GET /analytics/fraud/summary
Returns: { anomalies_this_week, flagged_trips_count, top_anomaly_type }

CACHING: Cache all analytics responses in Redis with 5-minute TTL
Cache key pattern: 'analytics:{endpoint}:{params_hash}'
Invalidate on new trip END or claim status change

=====
WEB — app/dashboard/admin/analytics/page.tsx
=====

Layout: 12-column grid, responsive

Row 1 — Summary Cards (4 cards):
[Total Employees] [Active Trips Now] [Claims This Month] [Total Reimbursed]
Each card: large number, subtitle, trend arrow (vs last period)
Color: blue / green / amber / purple
```

Row 2 – Charts:

Left (8 cols): BarChart – Daily trip count (last 30 days)
recharts: <BarChart data={...}><Bar dataKey='trip_count' fill='#1A56DB' />
X-axis: date labels (short: 'Jun 1'), Y-axis: count
Tooltip: date + count + total km

Right (4 cols): PieChart – Claims by status
APPROVED=green, REJECTED=red, SUBMITTED=amber, DRAFT=grey
Center label: total claim count

Row 3 – Monthly Reimbursement:

Full width LineChart: 6-month trend of approved claim amounts
Two lines: total_amount (blue) + count (dashed amber)
Dual Y-axis

Row 4 – Leaderboard:

Table: Rank | Employee | Trips | Total KM | Total Claimed
Period selector: This Week / This Month / This Year
Highlight top 3 rows with subtle gold/silver/bronze left border

Date Range Filter (global for page):

DateRangePicker component → updates all charts simultaneously
Pass date params as query strings to all analytics endpoints

LOADING STATES: Every chart shows a skeleton while fetching

ERROR STATES: If API fails, show 'Failed to load' with retry button

Master Development Checklist

Use this as your daily standup tracker. One row per task. Update status daily.

Module	Task	Assignee	Status	Sprint
M1	Monorepo scaffold + shared types	Lead	TODO	S1
M1	Zod schemas for all API inputs	Lead	TODO	S1
M2	Supabase Auth backend middleware	Backend	TODO	S1
M2	Mobile login + SecureStore	Mobile	TODO	S1
M2	Web login + middleware.ts RBAC	Web	TODO	S1
M3	Trip state machine + Haversine	Backend	TODO	S2
M3	Trip UI: punch-in/out/pause/resume	Mobile	TODO	S2
M4	Socket.IO server + Redis adapter	Backend	TODO	S2
M4	Mobile GPS emitter + offline queue	Mobile	TODO	S2
M4	Web live map + socket subscriber	Web	TODO	S2
M5	Claim auto-generate + approval API	Backend	TODO	S3
M5	Mobile claims screens	Mobile	TODO	S3
M5	Web claims queue + accountant view	Web	TODO	S3
M6	Admin user CRUD + Supabase sync	Backend	TODO	S4
M6	Admin panel UI + assignment view	Web	TODO	S4
M7	Mobile Mapbox trip map (live + replay)	Mobile	TODO	S2-3
M7	Web route replay map	Web	TODO	S3
M8	Fraud detection service + tests	Backend	TODO	S5
M8	Geofencing validation	Backend	TODO	S5
M9	Analytics aggregation endpoints	Backend	TODO	S4
M9	Analytics dashboard charts	Web	TODO	S4
CI/CD	GitHub Actions: lint+test+build	Lead	TODO	S6
Deploy	Backend to Render / ECS	Lead	TODO	S6
Deploy	Web to Vercel	Lead	TODO	S6
Deploy	Mobile: EAS Build + store submit	Mobile	TODO	S6

